

Schwarzschild Metric from Scratch via $\mathcal{UD}$ Compilation

Brian Beckman

Jan 2026

Abstract

In this notebook, we derive the Schwarzschild metric for the spacetime geometry in the vicinity of a black hole. The derivation is standard, but the novel contribution here is advancement of the $\mathcal{UD}$ calculus, moving its compiler from proof-of-concept to prototype.

The $\mathcal{UD}$ calculus automates symbolic operations such as covariant differentiation, tensor quotienting, canonicalizing indices, etc. The $\mathcal{UD}$ compiler translates between index notations in $\mathcal{UD}$ and component representations in Wolfram arrays.

- *A script version of this notebook, minus the prose, is available at https://github.com/rebcabin/RelativityToolkit/blob/main/schwarzschild_from_scratch.wl.*

Audience

This series of notebooks is targeted at readers with an undergraduate's STEM education—engineering, computer science, mathematics. Exposure to introductory courses in Classical Mechanics (e.g., [Goldstein](#)), Electrodynamics (e.g., [Jackson](#)), and Relativity (e.g., [Schutz](#)) will help.

- *With the exception of MTW, I do not put standard textbooks in the References section, but offer clickable (non-affiliate!) links instead in the prose of this notebook.*

Initialize the Relativity Toolkit

The toolkit is currently implemented as a Script rather than as a package for development speed and for transparency. That means that all symbols will be loaded into the Global namespace, including internal variables in Module constructs. This minor inconvenience will be mitigated in a future version.

Optional: Quit for a Fresh Kernel

- *Note, if you call `Quit` in a notebook, evaluation may hang. If you want to “Evaluate Notebook,” mark the following two cells “Evaluatable” in the Cell→Property menu, call `Quit` and*

FrontEndTokenExecute once each, then comment out the cells, or mark them unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can “Evaluate Notebook” in a clean environment. Otherwise, you can evaluate cells in this notebook individually.

```
In[ ]:= Quit[];

In[ ]:= FrontEndTokenExecute["DeleteGeneratedCells"]
```

Set Download Loci and Load Main Script

```
In[ ]:= RTbranch = "main";
RTbase = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
  RTbranch <> "/";
RTbase = "/Users/brian/Documents/GitHub/RelativityToolkit/";
RTbust = "?t=" <> ToString[Round[AbsoluteTime[]]];
RTbust = "";
(*Force fresh download*)
RTrules = RTbase <> "RelativityToolkit.wl" <> RTbust;
RTtests = RTbase <> "RelativityToolkit_RegressionTests.wl" <> RTbust;
RTviz = RTbase <> "RelativityToolkit_VisualShowcase.wl" <> RTbust;
Get[RTrules];

» Relativity Toolkit version : 1.8.0
» Relativity Connection Symbol :  $\Gamma$ 
```

Optional: See all Tests and Results from Prior Works

Make any of the following cells evaluatable if you wish. They’re unevaluatable by default to keep this notebook trim.

```
Get[RTtests];

In[ ]:= Get[RTviz] (* no semicolon, on purpose *)
In[ ]:= Get[RTbase <> "covariant_derivative.wl"]
In[ ]:= Get[RTbase <> "law_of_gamma.wl"]
Get[RTbase <> "connections_to_christoffels.wl"]
Get[RTbase <> "calculate_riemann.wl"]
```

Optional: Configure Relativity Connection Symbol

To properly compose covariant derivatives, $\mathcal{UD}$ must know the user’s reserved symbols for non-tensorial objects like connection coefficients. In gravitational physics, these force-producing connections are called Γ (capital gamma). Other theories like electrodynamics and Yang-Mills that can benefit from $\mathcal{UD}$ use other connection symbols.

We're going with Γ for this relativity/gravitation notebook, it's the default, so the following cell is optional.

```
In[ ]:= SetConnection[ $\Gamma$ ];
Relativity Engine: Connection set to  $\Gamma$ 
```

Solution Strategy

The question to answer in this notebook is “what is the vacuum spacetime geometry outside a static, spherically symmetric black hole?”

1. Guess at the form of the Schwarzschild metric tensor, $g_{\mu\nu}$, an *ansatz*.
 - *The story of Karl Schwarzschild's solution of the Einstein field equations while he was enlisted in the Prussian Army in WWI is fascinating. I was fortunate to have Schwarzschild's son, Martin, as one of my advisors in graduate school.*
2. Grind this form through new compiler components to generate Christoffel symbols, the Riemann curvature tensor $R^\mu{}_{\lambda\nu\rho}$, and the Ricci tensor $R_{\mu\nu}$, which is a specific contraction of Riemann.
3. Consult the Einstein field equations (not derived in this notebook):

$$R_{\mu\nu} - \frac{1}{2} R g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu} \quad (1)$$

where R is the metric trace of Ricci, namely $g^{\mu\nu} R_{\mu\nu}$.

4. Set the Einstein equations equal to zero for the vacuum ($T_{\mu\nu} = 0$, no stress-energy).
5. Contract the Einstein equations against $g^{\mu\nu}$ to get the metric trace:

$$g^{\mu\nu} R_{\mu\nu} - \frac{1}{2} R (g^{\mu\nu} g_{\mu\nu}) = R - \frac{1}{2} \times 4 R = 0 \implies R = 0 \quad (2)$$

6. Substitute this back into the Einstein equations to deduce that Ricci vanish in the vacuum:

$$R_{\mu\nu} - \frac{1}{2} (0) g_{\mu\nu} = 0 \implies R_{\mu\nu} = 0 \quad (3)$$

7. Derive an ODE for terms in the *ansatz* from vanishing Ricci components.
8. Invoke boundary conditions at infinity to solve the ODE, and invoke a Newtonian approximation far from the black hole to evaluate the constants of integration.
9. Compare against Wolfram's stored resource for the Schwarzschild metric.

Introducing New Compiler Functions

We have enhanced $\mathcal{UD}$ with several new functions in service of this task. The following sketches their functionality, and the rest of the notebook illustrates their use by example. These functions are sufficiently general that they will apply to other problems in general relativity and differential geometry in the future.

In this notebook, “matrix” is a synonym for a Wolfram 2D array. A tensor is either a matrix that satisfies coordinate-transformation rules, or a collection of symbolic $\mathcal{UD}$ expressions that also satisfy coordinate-transformation rules.

MatrixToUDRules

Converts the non-zero elements a matrix of components into rules for later application via `Replace` and `ReplaceAll` to symbolic expressions in $\mathcal{UD}$.

ChristoffelsFromMetric

Computes torsion-free Christoffel symbols from a symmetric metric tensor written in $\mathcal{UD}$ notation.

In[*]:= ContractAll

Contracts all bound indices in a $\mathcal{UD}$ expression over a given set of coordinate symbols.

In[*]:= EvaluateUDPartials

Converts partial derivatives in $\mathcal{UD}$ ’s comma notation into forms that Wolfram can manipulate.

In[*]:= MakeIndexer

Makes a function that can index a matrix of components via symbolic coordinates.

In[*]:= CalculateRiemannComponent

Given Christoffel symbols via an indexer, calculates a component of the Riemann curvature tensor by contracting over the Christoffel symbols.

In[*]:= CalculateRicciComponent

Given a Riemann tensor via an indexer, calculates a component of the Ricci tensor, a particular contraction of the Riemann tensor.

Ansatz

Set up a coordinate system in flat spacetime, at a safe distance from the central black hole, but with origin at that center. Measure all events with respect to this coordinate system. Solutions of these equations with respect to proper time account for spacetime curvature.

Assume as a guess, or *ansatz*, the following form for an element of proper distance along a geodesic:

$$ds^2 = -A(r) dt^2 + B(r) dr^2 + r^2 d\theta^2 + r^2 \sin^2(\theta) d\phi^2 \equiv g_{tt} dt^2 + g_{rr} dr^2 + g_{\theta\theta} d\theta^2 + g_{\phi\phi} d\phi^2 \quad (4)$$

with two unknown functions of radius, $A[r]$ and $B[r]$, and with symbolic coordinates, t , r , θ , ϕ , all assumed to be functions of proper time, τ , the time experienced by a particle in free-fall on a geodesic. Notice these functions do not depend on the angle coordinates θ and ϕ . That is the meaning of “spherically symmetric.”

Covariant Metric Tensor

Express the covariant metric tensor of this ansatz as a diagonal matrix (Wolfram array); save the coordinates for later:

```
In[8] := Echo[(SchwAnsatz = DiagonalMatrix[{-A[r], B[r], r^2, r^2 Sin[θ]^2}]) // MatrixForm,
  "Schw ansatz :"];
coords = {t, r, θ, ϕ};
» Schw ansatz : 
$$\begin{pmatrix} -A[r] & 0 & 0 & 0 \\ 0 & B[r] & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin[\theta]^2 \end{pmatrix}$$

```

- Coordinate time, t , is also a function of proper time, expressing time dilation.
- The tt component of $g_{\mu\nu}$ is negative because we employ the $(-1, 1, 1, 1)$ sign convention for Lorentz invariance.

Metric Rules

Call the new compiler function, **MatrixToUDRules**, to create some rules that map $\mathcal{UD}$ -indexed components of a given symbol, say g_{DD} , to the components of the ansatz matrix. We'll need these rules later to concretize an abstract expression for Γ .

```
In[9] := Echo[SchwRules = MatrixToUDRules[SchwAnsatz, gDD, d, coords], "Schw rules :"];
» Schw rules : {gDD t t → -A[r], gDD r r → B[r], gDD θ θ → r^2, gDD ϕ ϕ → r^2 Sin[θ]^2}
```

- For brevity, **MatrixToUDRules** doesn't produce rules for zeros in the matrix. We'll put those in later, by hand, in a concise way that exploits structural knowledge of g .
- TODO: Sparse-matrix version of **MatrixToUDRules** that concisely handles zero elements.

Contravariant Metric Tensor and Rules

The contravariant version, $g^{\mu\nu}$, of g is just the matrix inverse of the covariant version. Ending with the needed rules for g_{UU} , we get:

```
In[10] := SchwAnsatzInv = Inverse[SchwAnsatz];
Echo[SchwInvRules = MatrixToUDRules[SchwAnsatzInv, gUU, u, coords],
  "Schw Inv rules :"];
» Schw Inv rules : {gUU t t → -1/A[r], gUU r r → 1/B[r], gUU θ θ → 1/r^2, gUU ϕ ϕ → Csc[θ]^2/r^2}
```

Christoffel Symbols

The functions developed in an earlier notebook, *Riemann Curvature in $\mathcal{UD}$ + Compiler POC*, have been captured in a new compiler function, **ChristoffelsFromMetric**, which computes a commutator of covariant derivatives. See it as an unevaluated contraction over a bound **Unique** index:

```
In[11] := Echo[ChristoffelsFromMetric[gDD, gUU, σ, μ, ν], "Unevaluated Contraction :"];
» Unevaluated Contraction : 
$$\frac{1}{2} (g_{UU}^{\sigma \lambda 11}) (g_{DD \lambda 11 \mu, \nu} + g_{DD \lambda 11 \nu, \mu} - g_{DD \mu \nu, \lambda 11})$$

```

The new compiler function **ContractAll** exhibits the contraction carried out over all coordinates. Note

that the up-down index pairs are now the coordinate symbols rather than summation indices. Each term in the following exhibit is a product of components, not a further (recursive) contraction.

```
In[*]:= ContractAll[ChristoffelsFromMetric[gDD, gUU, σ, μ, ν] , coords]
Out[*]=
```

$$\begin{aligned} & \frac{1}{2} (g_{UU}^{\sigma r}) (g_{DD}^{r \mu, \nu}) + \frac{1}{2} (g_{UU}^{\sigma r}) (g_{DD}^{r \nu, \mu}) + \frac{1}{2} (g_{UU}^{\sigma t}) (g_{DD}^{t \mu, \nu}) + \\ & \frac{1}{2} (g_{UU}^{\sigma t}) (g_{DD}^{t \nu, \mu}) + \frac{1}{2} (g_{UU}^{\sigma \theta}) (g_{DD}^{\theta \mu, \nu}) + \frac{1}{2} (g_{UU}^{\sigma \theta}) (g_{DD}^{\theta \nu, \mu}) - \\ & \frac{1}{2} (g_{UU}^{\sigma r}) (g_{DD}^{\mu \nu, r}) - \frac{1}{2} (g_{UU}^{\sigma t}) (g_{DD}^{\mu \nu, t}) - \frac{1}{2} (g_{UU}^{\sigma \theta}) (g_{DD}^{\mu \nu, \theta}) - \\ & \frac{1}{2} (g_{UU}^{\sigma \phi}) (g_{DD}^{\mu \nu, \phi}) + \frac{1}{2} (g_{UU}^{\sigma \phi}) (g_{DD}^{\phi \mu, \nu}) + \frac{1}{2} (g_{UU}^{\sigma \phi}) (g_{DD}^{\phi \nu, \mu}) \end{aligned}$$

Inspect One Component

Now we see clearly the reason for the rules above:

```
In[*]:= Echo[TrttX = ContractAll[ChristoffelsFromMetric[gDD, gUU, r, t, t] , coords] /.
  SchwRules /.
  SchwInvRules, "Tr_{tt} ="];
» Tr_{tt} = -\frac{(-A[r])_{,r}}{2 B[r]} + \frac{1}{2} (g_{UU}^{r t}) (-A[r])_{,t} - \frac{1}{2} (g_{UU}^{r \theta}) (-A[r])_{,\theta} -
```

$$\frac{1}{2} (g_{UU}^{r \phi}) (-A[r])_{,\phi} + \frac{g_{DD}^{r t, t}}{B[r]} + (g_{UU}^{r \theta}) (g_{DD}^{\theta t, t}) + (g_{UU}^{r \phi}) (g_{DD}^{\phi t, t})$$

Notice the unevaluated partial derivatives in comma notation, $-A[r]_{,r}$ and $A[r]_{,t}$, $g_{DD}^{rt,t}$, etc.

Kill Off-Diagonal Elements

Write concise rules to zero-out the off-diagonal components:

```
In[*]:= offDiagonalRules = {
  gUU[u[a_], u[b_]] /; a != b -> 0,
  gDD[d[a_], d[b_]] /; a != b -> 0};
■ Recall that MatrixToUDRules does not currently do this, for practical reasons.
```

```
In[*]:= Echo[(TrttY = TrttX) /. offDiagonalRules, "Tr_{tt} diag only ="];
» Tr_{tt} diag only = \frac{0_{,t}}{B[r]} - \frac{(-A[r])_{,r}}{2 B[r]}
```

Notice, again, the unevaluated derivatives. $\mathcal{UD}$ doesn't evaluate derivatives until you tell it to do so.

- Killing the off-diagonal terms is not strictly necessary in this step as the next function, `EvaluateUDPartials`, would have zeroed them out anyway. But it's conceptually cleaner this way.

Evaluate $\mathcal{UD}$ Partial Derivatives

Another new $\mathcal{UD}$ compiler function, `EvaluateUDPartials`, produces a factor of $A'[r]$, which is, of

course, $A[r]_{,r}$, just in terms (the tick notation) that Wolfram understands. We'll wrestle this into differential equations for A .

```
In[*]:= Echo[ΓttA = EvaluateUDPartials[ΓttY], "Γtt evaluated ="];
```

$$\gg \Gamma_{tt}^r \text{ evaluated} = \frac{A'[r]}{2 B[r]}$$

All Γ Components

Now that we're happy with a reduction strategy, apply it to all components of Γ in a new 3D matrix, one dimension for each index:

```
In[*]:= (SchwΓ = Table[
  ContractAll[ChristoffelsFromMetric[gDD, gUU, σ, μ, ν], coords] /.
    SchwRules /. SchwInvRules /. offDiagonalRules // EvaluateUDPartials,
  {σ, coords}, {μ, coords}, {ν, coords}]) //
Echo[# // MatrixForm, "Ansatz Γ ="] &;
```

$$\gg \text{Ansatz } \Gamma = \begin{pmatrix} \begin{pmatrix} 0 \\ \frac{A'[r]}{2 A[r]} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} \frac{A'[r]}{2 A[r]} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} \frac{A'[r]}{2 B[r]} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{B'[r]}{2 B[r]} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ -\frac{r}{B[r]} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\frac{r \sin[\theta]^2}{B[r]} \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ \frac{1}{r} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\cos[\theta] \sin[\theta] \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{r} \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \cot[\theta] \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ \cot[\theta] \\ 0 \end{pmatrix} \end{pmatrix}$$

■ Here we really needed to kill the off-diagonal terms to produce a visually tolerable result.

Γ Indexer

To reduce opportunities for human error, we really want to index objects with the coordinates rather than with numerical position numbers. We'd rather write $\Gamma_{\text{get}}[r, t, t]$ than $\Gamma[[1, 0, 0]]$.

MakeIndexer is the new compiler function that makes a function that maps symbolic coordinates to Part expressions. Let's bind and test Γ_{get} on an arbitrary component:

```
In[*]:= Echo[(ΓGet = MakeIndexer[SchwΓ, coords])[r, r, r], "Γrr ="];
```

$$\gg \Gamma_{rr}^r = \frac{B'[r]}{2 B[r]}$$

The Riemann Ansatz in Full

Call another new compiler function, **CalculateRiemannComponent**, passing in Γ get and the coordinate symbols. Iterate symbolic indices μ, λ, ν , and ρ over all 256 combinations of the coordinates. There is far too much to inspect, by hand, here, but it's instructing to see some of it.

```
In[*]:= Echo[(SchwRiemann = Table[
  CalculateRiemannComponent[ $\mu$ ,  $\lambda$ ,  $\nu$ ,  $\rho$ ,  $\Gamma$ Get, coords],
  { $\mu$ , coords}, { $\lambda$ , coords}, { $\nu$ , coords}, { $\rho$ , coords}]] // Short[#, 5] &,
  "R $^{\mu}_{\lambda\nu\rho}$  ="];
» R $^{\mu}_{\lambda\nu\rho}$  = {{ {{ {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}},
  {{0, - $\frac{A[r] A'[r] B'[r] + B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 A[r]^2 B[r]}$ , 0, 0},
  {{ $\frac{A[r] A'[r] B'[r] + B[r] (<<1>>)}{4 A[r]^2 B[r]}$ , 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}}, {<<1>>},
  {{0, 0, 0,  $\frac{r \sin[\theta]^2 A'[r]}{2 A[r] \times B[r]}$ }, {0, 0, 0, 0}, {0, 0, 0, 0}, {- $\frac{r \sin[\theta]^2 A'[r]}{2 A[r] \times B[r]}$ , 0, 0, 0}},
  {{<<1>>}, {{<<1>>}, {<<1>>}, {<<1>>}, {<<1>>}}, {<<1>>}}
```

Riemann Indexer

As with Γ , we'll index Riemann via coordinate symbols. Look at a couple of components and check a famous antisymmetry condition:

```
In[*]:= Echo[(RGet = MakeIndexer[SchwRiemann, coords])[t, r, t, r], "R $^t_{ttr}$  ="];
» R $^t_{ttr}$  = - $\frac{A[r] A'[r] B'[r] + B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 A[r]^2 B[r]}$ 
In[*]:= Echo[RGet[t, r, r, t], "R $^r_{ttr}$  ="];
» R $^r_{ttr}$  =  $\frac{A[r] A'[r] B'[r] + B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 A[r]^2 B[r]}$ 
In[*]:= Echo[RGet[t, r, r, t] == -RGet[t, r, t, r]]
» True
Out[*]:= True
```

The Ricci Ansatz

Via another new compiler function, **CalculateRicciComponent**, which contracts Riemann over its up index and its second down index. Pass in R get and the coords, and iterate over the remaining 16 index values.


```
In[ ]:= Echo[ (SchwRicci = Table[
    CalculateRicciComponent[λ, ρ, RGet, coords],
    {λ, coords}, {ρ, coords}]] // TraditionalForm, "Rλρ ="];
» Rλρ =
```

$$\begin{pmatrix} \frac{r B(r) A'(r)^2 + A(r) (r A'(r) B'(r) - 2 B(r) (r A''(r) + 2 A'(r)))}{4 r A(r) B(r)^2} & 0 \\ 0 & -\frac{A(r) (r A'(r) + 4 A(r)) B'(r) + r B(r) (A'(r)^2 - 2 A(r) A''(r))}{4 r A(r)^2 B(r)} \\ 0 & 0 \\ 0 & 0 \end{pmatrix} \quad \frac{\frac{r A'(r)}{2 A(r)} + 1}{B(r)}$$

The Einstein Vacuum Equations

We can see immediately that there is a lot of structure in Ricci. In particular, the tt and rr components are very similar. We'll wrestle a tight, easy equation out of them.

Of course, we'll need a getter for Ricci.

```
In[ ]:= RicciGet = MakeIndexer[SchwRicci, coords];
Echo[Rtt = RicciGet[t, t], "Rtt ="];
Echo[Rrr = RicciGet[r, r], "Rrr ="];
» Rtt =
```

$$\frac{r B[r] A'[r]^2 + A[r] (r A'[r] B'[r] - 2 B[r] (2 A'[r] + r A''[r]))}{4 r A[r] B[r]^2}$$

```
» Rrr =
```

$$-\frac{A[r] (4 A[r] + r A'[r]) B'[r] + r B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 r A[r]^2 B[r]}$$

Expand R_{tt} and R_{rr} to spot a way to get rid of $A''[r]$:

```
In[ ]:= Rtt // Expand
Out[ ]:=
```

$$-\frac{A'[r]}{r B[r]} + \frac{A'[r]^2}{4 A[r] \times B[r]} + \frac{A'[r] B'[r]}{4 B[r]^2} - \frac{A''[r]}{2 B[r]}$$

```
In[ ]:= Rrr // Expand
Out[ ]:=
```

$$-\frac{A'[r]^2}{4 A[r]^2} - \frac{B'[r]}{r B[r]} - \frac{A'[r] B'[r]}{4 A[r] \times B[r]} + \frac{A''[r]}{2 A[r]}$$

It's obvious that $\frac{R_{tt}}{A[r]} + \frac{R_{rr}}{B[r]}$ will do it:

```
In[ ]:= (Rtt/A[r] + Rrr/B[r]) // Expand
Out[ ]:=
```

$$-\frac{A'[r]}{r A[r] \times B[r]} - \frac{B'[r]}{r B[r]^2}$$

Relation between $A[r]$ and $B[r]$

Because all these terms are 0 (Ricci vanishes in the vacuum), we can now solve for $B[r]$ in terms of $A[r]$:

```
In[*]:= Echo[DSolve[ $\frac{R_{tt}}{A[r]} + \frac{R_{rr}}{B[r]} == 0, B[r], r$ ], "Relation between A and B :"];
» Relation between A and B :  $\left\{ \left\{ B[r] \rightarrow \frac{c_1}{A[r]} \right\} \right\}$ 
```

Boundary Conditions at Infinity

Apply boundary conditions: flat space at infinity, meaning $\lim_{r \rightarrow \infty} A[r] = 1$ and $\lim_{r \rightarrow \infty} B[r] = 1$ (time and space have their undistorted coordinate meanings in flat space, far from the black hole), to deduce that $c_1 = 1$.

Final Equation and Solution

Now, drag out the $\theta\theta$ component and solve for a final differential equation for $A[r]$:

```
In[*]:= Echo[R $\theta\theta$  = RicciGet[ $\theta, \theta$ ], "R $\theta\theta$  ="];
» R $\theta\theta$  =  $-1 + \frac{1 + \frac{r A'[r]}{2 A[r]}}{B[r]} - \frac{r B'[r]}{2 B[r]^2}$ 
In[*]:= Echo[(finalODE = (R $\theta\theta$  == 0) /. {B[r] → 1/A[r], B'[r] → D[1/A[r], r]} //
FullSimplify), "Vacuum ODE for A[r] :"];
» Vacuum ODE for A[r] :  $A[r] + r A'[r] == 1$ 
In[*]:= Echo[SchwA = DSolve[finalODE, A[r], r], "Schwarzschild Solution :"];
» Schwarzschild Solution :  $\left\{ \left\{ A[r] \rightarrow 1 + \frac{c_1}{r} \right\} \right\}$ 
```

Constant of Integration

We just need to figure out the new integration constant, c_1 .

Demand the **Newtonian limit**: that the black hole behave like a normal Newtonian star when we are far away from it.

In this weak-field limit, it is a standard result (not derived in this notebook) that the tt component of the metric, g_{tt} , be directly related to the Newtonian gravitational potential Φ :

$$g_{tt} \approx -(1 + 2\Phi) \quad (5)$$

in geometric units where $G = c = 1$.

For a star of mass M , $\Phi = -M/r$, therefore

$$g_{tt} \approx -\left(1 - \frac{2M}{r}\right) \quad (6)$$

Substituting our solution for $A[r]$ into the ansatz, we have

$$g_{tt} = -A[r] = -\left(1 + \frac{c_1}{r}\right) \quad (7)$$

Thus $c_1 = -2M$.

Validation Against Wolfram Function Repository

Pulling together all our results:

```
In[ ]:= Echo[SchwAnsatz /. {B[r] -> 1/A[r]} /. SchwA /. {c1 -> -2 M},
  "Schwarzschild Solution via UD :"];
```

» Schwarzschild Solution via UD :

$$\left\{ \left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \sin[\theta]^2 \right\} \right\} \right\}$$

and comparing against the WFR resource

```
In[ ]:= Echo[ResourceFunction["MetricTensor"] ["Schwarzschild"] ["MatrixRepresentation"],
  "Schwarzschild Solution via WFR :"];
```

» Schwarzschild Solution via WFR :

$$\left\{ \left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \sin[\theta]^2 \right\} \right\} \right\}$$

we see a perfect match, except for WFR's convention of formal symbols (with underdots).

Conclusion

In this notebook, we derived the Schwarzschild metric for a static, spherically symmetric vacuum spacetime by constructing and employing a symbolic tensor calculator and compiler, *UD*.

The primary contribution of this work is the demonstration of the *UD* Toolkit (v1.8.0) as a viable prototype for automating calculations in general relativity. By separating the symbolic layer (abstract tensor notation, covariance, standard notations like comma, and Einstein summation) from the component layer (Wolfram arrays and differential equations), we achieved a readable and verifiable derivation.

Validation points include:

- **Symbolic Integrity:** The compiler handles index valence, canonicalization, and contraction via the `ContractAll` and `EvaluateUDPartials` functions.
- **Algebraic Simplification:** The toolkit generates the Riemann and Ricci tensors and decouples the vacuum field equations.
- **Ground Truth Verification:** The final derived solution, matches the standard result in the Wolfram Function Repository exactly.

Limitations

The current implementation operates as a script in the Global namespace. A future iteration (v2.0) will encapsulate the toolkit into a Wolfram Package to sanitize the namespace and offer a cleaner API.

Future Notebooks

With the engine now validated against the Schwarzschild solution, future notebooks will apply it to more complex geometries, including the computation of the Kretschmann scalar for physical singularities and the derivation of geodesic equations for orbital mechanics.

References

1. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
2. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
3. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
4. Beckman, Brian. *Covariant Derivatives and Connections in the UD Calculus*. Wolfram Community post.
5. Beckman, Brian. *Riemann Curvature in UD + Compiler POC*. Wolfram Community post (to appear).
6. Beckman, Brian. *Schwarzschild Orbits in the UD Calculus*. Wolfram Community post (to appear).
7. Wolfram Function Repository. *MetricTensor*. For ground truth.
8. Wolfram Function Repository. *ChristoffelSymbol*. For ground truth.
9. [MTW] Misner, Charles W.; Thorne, Kip S.; Wheeler, John Archibald. *Gravitation*. Princeton University Press, 2017.